

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Student Name	K. Hansikha
Experiment No.	PBL Part 2 – Task 8
Title	Express.js Middleware – Logging Incoming Requests

1. TITLE

Express.js Middleware – Logging Incoming Requests

2. AIM

To understand middleware in Express.js and implement custom middleware that logs incoming HTTP requests before passing control to the route handler.

3. OBJECTIVE

- To understand the purpose of Express.js middleware.
 - To create custom middleware.
 - To log HTTP method and request URL.
 - To use next() to pass control to the next middleware or route.
 - To configure an Express server and test it in a browser.
-

4. THEORY

Middleware in Express.js is a function that executes during the request-response cycle. It has access to the request object, response object, and the next() function. Middleware can be used for logging, authentication, validation, and other request-processing tasks. In this application, custom middleware logs the HTTP method and URL of every incoming request and then calls next() so the request can continue to the route handler.

5. ALGORITHM / PROCEDURE

1. Create a Node.js project.
 2. Install Express using npm install express.
 3. Import Express and create an application object.
 4. Define custom logging middleware with req, res, and next parameters.
 5. Log the current time, HTTP method, and request URL.
 6. Call next() to continue processing the request.
 7. Create a GET route for '/'.
 8. Start the server on port 3000.
 9. Open http://localhost:3000/ in a browser and observe the terminal log.
-

6. PROGRAM

package.json

```
{
  "name": "express-middleware-demo",
  "version": "1.0.0",
  "description": "Express.js logging middleware demonstration",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
}
```

```
"dependencies": {
  "express": "^5.1.0"
}
```

app.js

```
const express = require("express");

const app = express();
const PORT = 3000;

// Custom logging middleware
app.use((req, res, next) => {
  console.log(
    `${new Date().toLocaleString()} - ${req.method} ${req.url}`
  );
  next();
});

// Route
app.get("/", (req, res) => {
  res.send("Welcome to the Express.js Middleware Demo!");
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION

Code	Explanation
express()	Creates an Express application.
app.use()	Registers middleware that runs for incoming requests.
req.method	Provides the HTTP request method such as GET.
req.url	Provides the requested URL.
next()	Passes control to the next middleware or route handler.
app.get('/', ...)	Defines the response for a GET request to the root URL.
app.listen()	Starts the Express server on the specified port.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Test	Expected Result	Status
TC-01	GET /	Welcome message in browser and GET / in terminal	Pass
TC-02	Refresh /	A new log entry is generated	Pass

9. OUTPUT

Output 1: Program Output Screenshot



Welcome to the Express.js Middleware Demo!

10. RESULT

Thus, Express.js middleware was implemented successfully to log incoming HTTP requests and pass control to the route handler using `next()`.

How this would be submitted

Source Code

Task_8/

■■■ package.json

■■■ app.js

■■■ README.md